

# Ochrona Danych

## Laboratorium nr 5: Szyfr Rivesta-Shamira-Adlemana (RSA)

Jakub Jaśków 268416

20 stycznia 2026

### 1 Cel i zakres ćwiczenia

Celem laboratorium było praktyczne zapoznanie się z asymetrycznym algorytmem szyfrowania RSA. Zakres ćwiczenia obejmował implementację algorytmu w języku C++ (generowanie kluczy, szyfrowanie, deszyfrowanie), operacje na plikach binarnych oraz konfigurację bezpiecznego dostępu SSH do środowiska konteneryzowanego Docker przy użyciu kluczy asymetrycznych.

### 2 Zadanie 1 i 2: Implementacja RSA i operacje na plikach

#### 2.1 Opis implementacji

Na podstawie instrukcji przygotowano programy w języku C++ realizujące algorytm RSA.

- **Generowanie kluczy:** Liczby pierwsze  $p$  i  $q$  losowano z zakresu  $[16, 215]$ , co gwarantuje, że moduł  $n = p \cdot q$  nie przekracza wartości 65535 (mieści się na 16 bitach).
- **Klucz publiczny i prywatny:** Wyznaczono  $e$  względnie pierwsze z funkcją Eulera  $\phi(n)$  oraz  $d$  będące odwrotnością modularną  $e$ .
- **Szyfrowanie:** Zrealizowano funkcję szybkiego potęgowania modularnego dla operacji  $C = M^e \pmod{n}$ .

W Zadaniu 1 program szyfruje pojedynczy znak wczytany z klawiatury i wyświetla kryptogram w postaci szesnastkowej (HEX). W Zadaniu 2 program rozszerzono o obsługę plików binarnych. Szyfrowanie odbywa się "bajt po bajcie", gdzie każdy bajt pliku jawnego jest traktowany jako liczba  $M$ , szyfrowany do postaci  $C$ , a następnie zapisywany do pliku wyjściowego 'zaszyfr' (oraz analogicznie deszyfrowany do 'odszyfr').

#### 2.2 Wyniki działania programu

Poniższy zrzut ekranu prezentuje działanie programu: wprowadzenie znaku, wyświetlenie kluczy i kryptogramu (HEX) oraz komunikat o pomyślnym zaszyfrowaniu i odszyfrowaniu pliku. Sprawdzone poprawność operacji – plik odszyfrowany jest identyczny z plikiem wejściowym.



```

Processing triggers for libc-bin (2.35-0ubuntu3.11) ...
Processing triggers for ca-certificates (2024.02.03~22.04.1) ...
Updating certificates in /etc/ssl/certs...
0 added, 0 removed; done.
Running hooks in /etc/ca-certificates/update.d...
done.
root@4abce3845855:/# # Tworzymy u\305\274ytownika 'student' z has\305\202em 'student'
useradd -m -s /bin/bash student
echo 'student:student' | chpasswd
root@4abce3845855:/# # Startujemy SSH
service ssh start
* Starting OpenBSD Secure Shell server sshd
root@4abce3845855:/# exit
exit

```

Rysunek 3: Utworzenie użytkownika 'student', uruchomienie demona SSH i wyjście z kontenera.

### 3.3 Etap 3: Generowanie i transfer kluczy

Na systemie hosta wygenerowano parę kluczy RSA. Następnie, używając polecenia 'ssh-copy-id' (z wskazaniem odpowiedniego portu mapowanego z kontenera), przesłano klucz publiczny do autoryzowanych kluczy użytkownika 'student' na kontenerze.

```

mango@T14:~/.ssh$ ssh-keygen -t rsa -f id_rsa_lab
Generating public/private rsa key pair.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in id_rsa_lab
Your public key has been saved in id_rsa_lab.pub
The key fingerprint is:
SHA256:7c1ks1+B6qRxfX8BavQgpbvbo2c+jpo6y8nmcrdCbS0 mango@T14
The key's randomart image is:
+---[RSA 3072]-----+
|
| ..|
| ...o ..|
| ..o*..|
| +=o...|
| .S...=o o|
| o E .+ =oo..|
| + = .. +. +|
| . O o. o+. .|
| =.++oo=o..|
+---[SHA256]-----+
mango@T14:~/.ssh$ ssh-copy-id -p 2222 student@localhost
The authenticity of host '[localhost]:2222 ([127.0.0.1]:2222)' can't be established.
ED25519 key fingerprint is SHA256:vqo9VjpHrSFUP9JaSQ33CHY9BuiJ5qbjvc38Ea17NPo.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 2 key(s) remain to be installed -- if you are prompted now it is to install the new keys
student@localhost's password:
Permission denied, please try again.
student@localhost's password:

Number of key(s) added: 2

Now try logging into the machine, with: "ssh -p '2222' 'student@localhost'"
and check to make sure that only the key(s) you wanted were added.

mango@T14:~/.ssh$ ssh -p 2222 student@localhost
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-90-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

```

Rysunek 4: Generowanie kluczy RSA na hoście i kopiowanie ich do kontenera.

### 3.4 Etap 4: Weryfikacja połączenia

Nawiązano połączenie SSH z kontenerem bez użycia hasła (logowanie kluczem). W celu ostatecznej weryfikacji wyświetlono zawartość klucza publicznego na hoście oraz pliku `authorized_keys` wewnątrz kontenera, potwierdzając ich tożsamość.

```
mango@T14:~/ssh$ ssh -p 2222 student@localhost
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-98-generic x86_64)

+ Documentation:  https://help.ubuntu.com
+ Management:    https://landscape.canonical.com
+ Support:        https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.
student@4abce3845855:~$ ls -l ~/.ssh
authorized_keys
student@4abce3845855:~$ cat ~/.ssh/authorized_keys
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQGCv8ktx6UCDVuS0aXid1SMxPm99YbcmH/nrXN4Fs/rK1CkByc+Z/PrR/0ocY0YnK3qWxkma1Sa6j6F8A69MhrbUD2681nvxdi+VR9yglY5ip0bQ8I0shcC3rpeCj3+UjRit0WjF4DjXqH5R0ZKQVr
+tbKE0ND0F0ebCh6vvgZ8A0jBJrVdFYUACvUGvKXTzQL0HR8Hw5xq8jql3P+abJLta4K4tBnUdsyS5x1KFYY/SSLLmV4wBrqHrn9a5076bHiiHmLt4yQ+J808AVP6MTbkf071cxp6QRj5vSIFTnw8g9SpyM0BHTus2XIV48FRXEHCrqgnodkkyK
xyRIU1iV4hsqI0t0nkivHedwJtpTtQys5W8M+gD+qNeHU3QT6w/QNP1QJ2VYkMVPxyLtx3wQ1+p0u6nYEvQd7ZGK0WHEUIm5gA0Xm/nFF1e624K2aacx9VuwLr3kjdsdsU8aulP0KFLIbsJP1NV6XcNkiWQNFcxYVvh/hzf0uS0= mango@T14
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQGCv8ktx6UCDVuS0aXid1SMxPm99YbcmH/nrXN4Fs/rK1CkByc+Z/PrR/0ocY0YnK3qWxkma1Sa6j6F8A69MhrbUD2681nvxdi+VR9yglY5ip0bQ8I0shcC3rpeCj3+UjRit0WjF4DjXqH5R0ZKQVr
+tbKE0ND0F0ebCh6vvgZ8A0jBJrVdFYUACvUGvKXTzQL0HR8Hw5xq8jql3P+abJLta4K4tBnUdsyS5x1KFYY/SSLLmV4wBrqHrn9a5076bHiiHmLt4yQ+J808AVP6MTbkf071cxp6QRj5vSIFTnw8g9SpyM0BHTus2XIV48FRXEHCrqgnodkkyK
xyRIU1iV4hsqI0t0nkivHedwJtpTtQys5W8M+gD+qNeHU3QT6w/QNP1QJ2VYkMVPxyLtx3wQ1+p0u6nYEvQd7ZGK0WHEUIm5gA0Xm/nFF1e624K2aacx9VuwLr3kjdsdsU8aulP0KFLIbsJP1NV6XcNkiWQNFcxYVvh/hzf0uS0= mango@T14
mango@T14:~/ssh$
```

Rysunek 5: Udatne logowanie SSH oraz porównanie kluczy publicznych (host vs kontener).

## 4 Wnioski

1. Zaimplementowany algorytm RSA poprawnie realizuje proces szyfrowania i deszyfrowania danych, co potwierdzono na przykładzie plików binarnych.
2. Wykorzystanie Dockera pozwoliło na bezpieczną i izolowaną symulację serwera SSH.
3. Konfiguracja logowania przy użyciu pary kluczy RSA (Zadanie 3) znacznie podnosi poziom bezpieczeństwa w porównaniu do standardowego uwierzytelniania hasłem, eliminując ryzyko jego przechwycenia lub złamania metodą siłową. Proces ten zautomatyzowano przy użyciu narzędzi `ssh-keygen` oraz `ssh-copy-id`.